

# VELORA — Module Inventory (Part 2)

Repositories · DTOs · Form Requests · Resources

## 4. Repository Interfaces

app/Modules/Inventory/Repositories/Contracts/WarehouseRepositoryInterface.php

```
<?php
```

```
namespace App\Modules\Inventory\Repositories\Contracts;
```

```
use App\Modules\Inventory\Models\Warehouse;
```

```
use Illuminate\Database\Eloquent\Collection;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
interface WarehouseRepositoryInterface
```

```
{  
    public function findById(string $id): ?Warehouse;  
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator;  
    public function allActive(string $businessId): Collection;  
    public function create(array $data): Warehouse;  
    public function update(string $id, array $data): Warehouse;  
    public function delete(string $id): bool;  
    public function setMainWarehouse(string $warehouseId, string $businessId): void;  
}
```

## app/Modules/Inventory/Repositories/Contracts/StockMovementRepositoryInterface.php

```
<?php
```

```
namespace App\Modules\Inventory\Repositories\Contracts;
```

```
use App\Modules\Inventory\Models\StockMovement;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
interface StockMovementRepositoryInterface
```

```
{  
    public function findById(string $id): ?StockMovement;  
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator;  
    public function create(array $data): StockMovement;  
    public function updateStatus(string $id, string $status): StockMovement;  
  
    // Items  
    public function createItem(array $data): \App\Modules\Inventory\Models\StockMovementItem;  
}
```

## 5. Repository Implementations

app/Modules/Inventory/Repositories/Eloquent/WarehouseRepository.php

```
<?php
```

```
namespace App\Modules\Inventory\Repositories\Eloquent;
```

```
use App\Modules\Inventory\Models\Warehouse;
```

```
use App\Modules\Inventory\Repositories\Contracts\WarehouseRepositoryInterface;
```

```
use App\Shared\Repositories\BaseRepository;
```

```
use Illuminate\Database\Eloquent\Collection;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
use Illuminate\Support\Facades\DB;
```

```
class WarehouseRepository extends BaseRepository implements WarehouseRepositoryInterface
{
```

```
    public function __construct(Warehouse $model)
    {
        parent::__construct($model);
    }
```

```
    public function findById(string $id): ?Warehouse
    {
        return Warehouse::with('branch')->find($id);
    }
```

```
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator
    {
        return Warehouse::where('business_id', $businessId)
            ->when($filters['branch_id'] ?? null, fn($q, $v) => $q->where('branch_id', $v))
            ->when($filters['search'] ?? null, fn($q, $v) =>
                $q->where(fn($q) => $q->where('name', 'like', "%{$v}%")->orWhere('code', 'like', "%{$v}%"))
            )
            ->with('branch')
            ->orderByDesc('is_main')
            ->latest()
            ->paginate($filters['per_page'] ?? 15);
    }
```

```
    public function allActive(string $businessId): Collection
    {
        return Warehouse::where('business_id', $businessId)
            ->active()
            ->orderByDesc('is_main')
            ->orderBy('name')
            ->get();
    }
```

```
    public function create(array $data): Warehouse
    {
        return Warehouse::create($data);
    }
```

```

public function update(string $id, array $data): Warehouse
{
    $warehouse = Warehouse::findOrFail($id);
    $warehouse->update($data);
    return $warehouse->fresh(['branch']);
}

public function delete(string $id): bool
{
    return Warehouse::findOrFail($id)->delete();
}

public function setMainWarehouse(string $warehouseId, string $businessId): void
{
    DB::transaction(function () use ($warehouseId, $businessId) {
        Warehouse::where('business_id', $businessId)->update(['is_main' => false]);
        Warehouse::where('id', $warehouseId)->update(['is_main' => true]);
    });
}
}

```

## app/Modules/Inventory/Repositories/Eloquent/StockMovementRepository.php

```
<?php
```

```
namespace App\Modules\Inventory\Repositories\Eloquent;

use App\Modules\Inventory\Models\StockMovement;
use App\Modules\Inventory\Models\StockMovementItem;
use App\Modules\Inventory\Repositories\Contracts\StockMovementRepositoryInterface;
use App\Shared\Repositories\BaseRepository;
use Illuminate\Pagination\LengthAwarePaginator;

class StockMovementRepository extends BaseRepository implements StockMovementRepositoryInterface
{
    public function __construct(StockMovement $model)
    {
        parent::__construct($model);
    }

    public function findById(string $id): ?StockMovement
    {
        return StockMovement::with(['warehouse', 'user', 'items.product', 'items.variant'])->find($id);
    }

    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator
    {
        return StockMovement::where('business_id', $businessId)
            ->when($filters['warehouse_id'] ?? null, fn($q, $v) => $q->where('warehouse_id', $v))
            ->when($filters['type'] ?? null, fn($q, $v) => $q->where('type', $v))
            ->when($filters['status'] ?? null, fn($q, $v) => $q->where('status', $v))
            ->when($filters['search'] ?? null, fn($q, $v) =>
                $q->where('reference_number', 'like', "%{$v}%")
            )
            ->with(['warehouse', 'user'])
            ->latest()
            ->paginate($filters['per_page'] ?? 15);
    }

    public function create(array $data): StockMovement
    {
        return StockMovement::create($data);
    }

    public function updateStatus(string $id, string $status): StockMovement
    {
        $movement = StockMovement::findOrFail($id);
        $movement->update(['status' => $status]);
        return $movement->fresh();
    }

    public function createItem(array $data): StockMovementItem
    {
        return StockMovementItem::create($data);
    }
}
```

```
}  
}
```

## 6. Register Repository Bindings

```
// Tambahkan ke app/Providers/RepositoryServiceProvider.php
```

```
use App\Modules\Inventory\Repositories\Contracts\WarehouseRepositoryInterface;  
use App\Modules\Inventory\Repositories\Eloquent\WarehouseRepository;  
use App\Modules\Inventory\Repositories\Contracts\StockMovementRepositoryInterface;  
use App\Modules\Inventory\Repositories\Eloquent\StockMovementRepository;  
  
$this->app->bind(WarehouseRepositoryInterface::class, WarehouseRepository::class);  
$this->app->bind(StockMovementRepositoryInterface::class, StockMovementRepository::class);
```

## 7. DTOs

### app/Modules/Inventory/DTOs/CreateWarehouseDTO.php

```
<?php

namespace App\Modules\Inventory\DTOs;

class CreateWarehouseDTO
{
    public function __construct(
        public readonly string $businessId,
        public readonly string $name,
        public readonly ?string $branchId = null,
        public readonly ?string $code     = null,
        public readonly ?string $address  = null,
        public readonly bool    $isMain   = false,
    ) {}

    public static function fromRequest(array $data, string $businessId): static
    {
        return new static(
            businessId: $businessId,
            name:       $data['name'],
            branchId:   $data['branch_id'] ?? null,
            code:       $data['code']      ?? null,
            address:    $data['address']   ?? null,
            isMain:     $data['is_main']   ?? false,
        );
    }

    public function toArray(): array
    {
        return [
            'business_id' => $this->businessId,
            'branch_id'   => $this->branchId,
            'name'        => $this->name,
            'code'        => $this->code,
            'address'     => $this->address,
            'is_main'     => $this->isMain,
        ];
    }
}
```

## app/Modules/Inventory/DTOs/CreateStockMovementDTO.php

```
<?php
```

```
namespace App\Modules\Inventory\DTOs;
```

```
class CreateStockMovementDTO
```

```
{
    public function __construct(
        public readonly string $businessId,
        public readonly string $warehouseId,
        public readonly string $userId,
        public readonly string $type,
        public readonly string $status,
        public readonly ?string $notes = null,
        public readonly ?string $referenceId = null,
        public readonly ?string $referenceType = null,
        public readonly array $items = [],
    ) {}

    public static function fromRequest(array $data, string $businessId, string $userId): static
    {
        return new static(
            businessId: $businessId,
            warehouseId: $data['warehouse_id'],
            userId: $userId,
            type: $data['type'],
            status: $data['status'] ?? 'completed',
            notes: $data['notes'] ?? null,
            referenceId: $data['reference_id'] ?? null,
            referenceType: $data['reference_type'] ?? null,
            items: $data['items'] ?? [],
        );
    }
}
```



## 8. Form Requests

app/Modules/Inventory/Requests/StoreWarehouseRequest.php

```
<?php

namespace App\Modules\Inventory\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class StoreWarehouseRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        $businessId = auth()->user()->business_id;

        return [
            'name' => ['required', 'string', 'max:100'],
            'branch_id' => ['nullable', 'uuid', Rule::exists('branches', 'id')->where('business_id', $businessId)],
            'code' => ['nullable', 'string', 'max:20', Rule::unique('warehouses')->where('business_id', $businessId)],
            'address' => ['nullable', 'string'],
            'is_main' => ['nullable', 'boolean'],
        ];
    }
}
```

## app/Modules/Inventory/Requests/StoreStockMovementRequest.php

```
<?php
```

```
namespace App\Modules\Inventory\Requests;
```

```
use App\Modules\Inventory\Enums\MovementStatus;
```

```
use App\Modules\Inventory\Enums\MovementType;
```

```
use Illuminate\Foundation\Http\FormRequest;
```

```
use Illuminate\Validation\Rule;
```

```
use Illuminate\Validation\Rules\Enum;
```

```
class StoreStockMovementRequest extends FormRequest
```

```
{
```

```
    public function authorize(): bool { return true; }
```

```
    public function rules(): array
```

```
    {
```

```
        $businessId = auth()->user()->business_id;
```

```
        return [
```

```
            'warehouse_id' => ['required', 'uuid', Rule::exists('warehouses', 'id')->where('business_id', $businessId)],
```

```
            'type' => ['required', new Enum(MovementType::class)],
```

```
            'status' => ['nullable', new Enum(MovementStatus::class)],
```

```
            'notes' => ['nullable', 'string'],
```

```
            'reference_id' => ['nullable', 'uuid'],
```

```
            'reference_type' => ['nullable', 'string', 'max:50'],
```

```
            'items' => ['required', 'array', 'min:1'],
```

```
            'items.*.product_id' => ['required', 'uuid', Rule::exists('products', 'id')->where('business_id', $businessId)],
```

```
            'items.*.variant_id' => ['nullable', 'uuid', Rule::exists('product_variants', 'id')->where('business_id', $businessId)],
```

```
            'items.*.quantity' => ['required', 'integer', 'min:1'], // Always absolute value
```

```
            'items.*.cost_price' => ['nullable', 'integer', 'min:0'],
```

```
            'items.*.notes' => ['nullable', 'string'],
```

```
        ];
```

```
    }
```

```
}
```

## 9. Resources

app/Modules/Inventory/Resources/WarehouseResource.php

```
<?php

namespace App\Modules\Inventory\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class WarehouseResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'          => $this->id,
            'name'         => $this->name,
            'code'         => $this->code,
            'address'      => $this->address,
            'is_main'      => $this->is_main,
            'is_active'    => $this->is_active,
            'branch'       => $this->whenLoaded('branch', fn() => [
                'id' => $this->branch->id,
                'name' => $this->branch->name,
            ]),
            'created_at' => $this->created_at?->toIso8601String(),
        ];
    }
}
```

## app/Modules/Inventory/Resources/StockMovementResource.php

```
<?php
```

```
namespace App\Modules\Inventory\Resources;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Http\Resources\Json\JsonResource;
```

```
class StockMovementResource extends JsonResource
```

```
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'reference_number' => $this->reference_number,
            'type' => $this->type,
            'type_label' => $this->type->label(),
            'status' => $this->status,
            'status_label' => $this->status->label(),
            'notes' => $this->notes,
            'reference_id' => $this->reference_id,
            'reference_type' => $this->reference_type,

            'warehouse' => new WarehouseResource($this->whenLoaded('warehouse')),
            'user' => $this->whenLoaded('user', fn() => [
                'id' => $this->user->id,
                'name' => $this->user->name,
            ]),

            'items' => $this->whenLoaded('items', function() {
                return $this->items->map(fn($item) => [
                    'id' => $item->id,
                    'product_id' => $item->product_id,
                    'variant_id' => $item->product_variant_id,
                    'product_name' => $item->product->name ?? null,
                    'variant_name' => $item->variant->name ?? null,
                    'quantity' => $item->quantity,
                    'before_quantity' => $item->before_quantity,
                    'after_quantity' => $item->after_quantity,
                    'cost_price' => $item->cost_price,
                    'notes' => $item->notes,
                ]);
            }),

            'created_at' => $this->created_at?->toIso8601String(),
        ];
    }
}
```